Josh Bottelberghe

Go / Distributed-Systems Engineer, specialized in networking at scale

Vancouver, WA (remote) · jbottelb@gmail.com · linkedin.com/in/josh-bottelberghe · github.com/joshbottelberghe · joshbottelberghe.github.io

SUMMARY

Software engineer, ~4.5 years, mostly Go. I own the route-visualization system at Alkira, a multi-cloud pipeline that turns live routing state off a fleet of nodes into a per-tenant view for 200+ enterprise customers, under a sub-5-minute freshness target, with zero major outages. I rebuilt its compute engine three times as it scaled and made recompute cost track the change, not the size of the state; two patents came out of that work. I go deep on BGP/FRR, I measure before I optimize, and I build tooling the rest of the team uses.

SKILLS

Languages: Go (primary), Python, Bash, C/C++

Distributed systems: high-scale pipelines, caching & cache invalidation, concurrency, leader election, sharding, backpressure, consistency (live vs. provisioned state)

Infrastructure: AWS (S3 at scale), Redis, Kubernetes (statefulsets, leader election), etcd

Observability: Prometheus, Grafana, structured logging & tracing

Networking: BGP, FRR/Zebra internals, route policy, multi-cloud routing, NAT

Testing / CI: production-replay soak testing, scale harnesses, dependency security (govulncheck, gitleaks)

EXPERIENCE

Senior Software Engineer, Alkira, Inc. — 2022 to Present

Multi-cloud network-as-a-service (acquired by Lumen Technologies, 2026) · Remote · Senior Software Engineer 2024 to Present, Software Engineer 2022 to 2024

- Own the route-visualization pipeline: nodes across AWS, Azure, and GCP export routes to S3; a sharded, leader-elected compute tier reads them, builds the per-tenant view, and caches it in Redis for the customer APIs. 200+ tenants, sub-5-minute freshness, no major outages.
- Rebuilt the compute engine three times as it scaled: per-tenant threads, then a Redis-backed cache, then segment-level partial compute (recompute only the segment that changed). Cut cache-update time ~74% and CPU ~64%. This work is the basis of one of my two patents.
- Wrote a soak-test harness in Go that replays recorded production traffic against two branches and reports the difference in CPU, memory, and latency, so optimization calls are measured, not guessed. It shipped the wins and caught a regression before it merged.
- Ran a release-targeted program to cut wasted recomputes: content-hashing, dependency tracking, and a dirty-set that recomputes only the segments a change reaches. Removed ~90% of redundant recomputes and the no-op recomputes that were blocking real updates.
- Cut cache memory ~40% with a struct-encoding change; parallelized compute to take multi-minute updates to seconds; tracked down multi-hour S3 stalls (a hung ListObjects, clock skew) and fixed them with timeouts and pagination.
- Built a route-intelligence tooling server in Go (rewrote it to a single binary, v1 to v2.1 in ~2 weeks) that the team uses for on-call and route history. Standardized CI and dependency security across 18 Go repos, fixing real CVEs and an IDOR along the way.
- Root-caused hard BGP/FRR production bugs: nondeterministic route ordering (Go map iteration), a stale export hint, and a live-vs-provisioned read that surfaced un-provisioned state.
- Owned features end-to-end across Product and QA, mentored engineers, wrote an 11-lesson networking curriculum, and became the team's go-to on the domain.

Earlier: University of Notre Dame and internships — 2020 to 2022

Software Development Intern (RPA automation, Python) · TA, Data Structures and Fundamentals of Computing · Stanford *Code in Place* Section Leader (led a Python cohort).

PATENTS

Two patents, filed 2025: one on route visualization in general, and one on two route-processing methods I designed to compute in proportion to change, not total state.

PROJECTS

Cryptocurrency from scratch (Python): a working blockchain: full nodes, a proof-of-work miner, public-key wallets, and a custom peer-to-peer message protocol. The distributed-systems project I walked through in my interviews, and what got me the role at Alkira. github.com/joshbottelberghe/crypto

Neural machine translation with transformers: built self-attention encoder-decoder (sequence-to-sequence) models for language translation.

EDUCATION

B.S. Computer Science, University of Notre Dame (2022) · Major GPA 3.7. Coursework: Distributed Systems, Operating Systems, Computer Networks, Databases, Machine Learning, NLP.

BEYOND WORK

NCAA Division I swimmer at Notre Dame: team captain, still hold the school records in the 100 and 200 breaststroke, and All-American. Outside work I play guitar and drums (I record at home), cook on cast iron, spend time with my cats and my loving girlfriend, and game.